Ron Hensley

ronkhensley@gmail.com

AAS in AI Software Engineering

PY102

Challenge: making an algorithm more efficient

Today
Welcome to your next lesson: Challenge: making an algorithm more efficient.

I'm preparing your personalized lesson based on what you've learned, what's next, and the best way to help you grow.

Let’s dive in — you’re making real progress.

Ron, let’s jump in with one clean idea: spotting wasted work in an algorithm 💡

Imagine this Python snippet that prints all pairs of players in a lobby:

Python

```
players = ["Ana", "Ben", "Cara", "Dan"]

for p1 in players:
    for p2 in players:
        if p1 != p2:
            print(p1, "vs", p2)
```

This prints both Ana vs Ben and Ben vs Ana. For many games, that’s duplicate work.

If we only care about each unique pair once (order doesn’t matter), does this code feel:

A) Fine as-is, not really wasteful
B) Clearly doing extra, avoidable work

Just answer A or B and we’ll use it as our first “efficiency boss battle” 😄

b

Nice, I agree with you: B is right — it’s clearly doing extra work ✨

That’s our first key idea in efficiency:

If an algorithm does the same logical job more than once, there’s usually a way to make it faster.

For the pairs example, we could avoid duplicates by only pairing each player with the ones that come after it in the list (so we get Ana vs Ben but not Ben vs Ana).

Before I show code, quick check:

Are you comfortable with basic for loops over lists in Python, or want a 30□second refresher first?

I’m comfortable with loops

Perfect 👍 That lets us move a bit faster.

Let’s start with a super simple “find the waste” pattern.

Look at this Python code:

Python

```
numbers = [3, 5, 7, 9]

total = 0
for n in numbers:
    total = total + sum(numbers)

print(total)
```

Step by step:

sum(numbers) is calculated inside the loop.
But numbers never changes.

So sum(numbers) is the same value every time, but we recompute it on every loop 🔁

In your own words (short is fine):

What’s one simple change you could make to avoid repeating the same sum(numbers) work over and over?

total += n

Nice thought, Ron — that would make the result more sensible, but it’s a different fix than the one I’m targeting here 👍

Right now the code is:

Python

```
numbers = [3, 5, 7, 9]

total = 0
for n in numbers:
    total = total + sum(numbers)
```

The wasted work is that sum(numbers) is recomputed every loop even though it never changes.

A classic efficiency move is:

Compute that expensive thing once before the loop
Reuse the stored result inside the loop

Visually:

Python

```
# bad pattern
for ...:
    repeated_work()

# better pattern
cached = repeated_work()
for ...:
    use(cached)
```

Make sense why moving sum(numbers) out of the loop removes the repeated work? 🎯

If yes, I'll give you a tiny code challenge in the editor to do exactly that.

Yes, that makes sense

Great ☑

Here's the key pattern we'll practice in this lesson:

If a value doesn't change, don't recompute it in a loop. Compute once, then reuse.

For our example, the idea is:

Python

```
numbers = [3, 5, 7, 9]

sum_numbers = sum(numbers)   # compute once ☑

total = 0
for n in numbers:
    total = total + sum_numbers   # reuse, no repeated sum() calls
```

Your turn (in the code editor) 🖋

Open the code editor (</>).
Start from the original version.
Change it so sum(numbers) is computed before the loop and reused inside.
Run, then Submit code.

Once you've submitted, tell me and we'll do the next efficiency "boss battle" 🥊

oh now i get last one i though the error was duplicate adding of the sum you actuay wanted to re-add the sum every time just to not have to compute it each time yes il try it now

Love that clarification, Ron — you've got it exactly now 🎯

We do want to add that same sum multiple times (that's what the original logic was doing).
We don't want to recompute the sum multiple times, because that's wasted work.

So the "efficiency win" is: same behavior, fewer repeated calculations.

Go ahead and try your fix in the editor now:

compute sum(numbers) once before the loop,
reuse that stored value inside the loop,
then Run → Submit code.

I'll check it when it comes through and then we'll tackle the next efficiency boss 👾

Python

```
numbers = [3, 5, 7, 9]

print("Before change")
total = 0
for n in numbers:
    total = total + sum(numbers)
print(total)

print("After change")
```

```
sum_numbers = sum(numbers)   # compute once ☑
total = 0
for n in numbers:
    total = total + sum_numbers   # reuse, no repeated sum() calls
print(total)
```

Nice work, Ron 🔥

You did exactly what we wanted:

Before change: sum(numbers) runs once per loop → 4 times total
After change: sum(numbers) runs once before the loop → 1 time total
Output stays the same (96) ☑

So: same result, less work → more efficient algorithm.

Next step, we'll look at a different kind of wasted work: using a loop where we could use a direct lookup instead.

Ready for the next efficiency boss battle? 👾

Yes, bring on the next challenge!

Alright, boss battle #2 coming up 👾

Imagine you have a list of players and their scores, and you want to print the score for each requested name:

Python

```
players = ["Ana", "Ben", "Cara", "Dan"]
scores = [10, 15, 8, 12]

requested = ["Cara", "Ana", "Dan"]

for name in requested:
    for i in range(len(players)):
        if players[i] == name:
            print(name, "has score", scores[i])
```

Step-by-step:

For each name in requested, we loop over all players.
That inner loop is a repeated scan of the same list.

This works, but it's doing a lot of repeated searching.

Concept for this round (don't code yet):

A dictionary can give you direct lookup by name, instead of scanning the whole players list every time.

Does the idea "build a dict once, then use it inside the loop" feel clear, or want a 20□second recap of how dicts work in Python before we refactor?

I understand the idea

Nice 💪 That's exactly the idea we need here.

Let's walk the refactor in two clear steps:

Before the main loop, build a dict that maps each player to their score.
Then, for each name in requested, just do a direct lookup in that dict instead of

looping over players each time.

Visually:

Python

```
# Step 1: build once
score_by_name = {
    "Ana": 10,
    "Ben": 15,
    "Cara": 8,
    "Dan": 12
}

# Step 2: reuse
for name in requested:
    print(name, "has score", score_by_name[name])
```

That kills the nested loop and turns “scan whole list every time” into “jump straight to the answer”.

Your turn in the editor ✏

Start from this version:

Python

```
players = ["Ana", "Ben", "Cara", "Dan"]
scores = [10, 15, 8, 12]

requested = ["Cara", "Ana", "Dan"]

for name in requested:
    for i in range(len(players)):
        if players[i] == name:
            print(name, "has score", scores[i])
```

Change it so:
You create a dict once before the for name in requested loop.
Inside that loop, you do a direct lookup instead of the inner for i in range(...) loop.
The printed output stays the same.

Run it, then Submit code.

I’ll check it and then we’ll talk about how this changes the Big O (efficiency class) 😊

Python

```
players = ["Ana", "Ben", "Cara", "Dan"]
scores = [10, 15, 8, 12]

requested = ["Cara", "Ana", "Dan"]

for name in requested:
    for i in range(len(players)):
        if players[i] == name:
            print(name, "has score", scores[i])
i = 0
my_dict = {}
for player in players:
    my_dict[player] = scores[i]
    i += 1
print(my_dict)

for name, score in my_dict.items():
    if name in requested:
            print(f'{name} has score {score}')
```

Nice job, Ron 👏 You did two good things here:

You built a dict that maps names to scores:

Python

```
my_dict = {}
i = 0
for player in players:
    my_dict[player] = scores[i]
    i += 1
```

You then used that mapping to print scores.

That’s the right idea: build something once, then reuse it so you don’t have to scan the list each time.

The only tweak I’d make: we don’t need to loop over my_dict.items() and check if name in requested. That’s a bit like going back to scanning everyone again 🙂

We can keep it even closer to the original logic:

Python

```
players = ["Ana", "Ben", "Cara", "Dan"]
scores = [10, 15, 8, 12]

requested = ["Cara", "Ana", "Dan"]

# build dict once
score_by_name = {}
for i in range(len(players)):
    score_by_name[players[i]] = scores[i]

# direct lookup, no nested loop
for name in requested:
    print(name, "has score", score_by_name[name])
```

One loop to build the dict, one loop to answer all requests. No nested loops at all.

Question for you (just conceptually, no code):

Compared to the original version with the nested loops, does this new dict-based approach:

still do the same job (print the same kind of info)?
do less repeated work to get there?

Once you answer that, we’ll push to the next efficiency challenge with nested loops and Big O ✡

less repeated work

Exactly 💯 — same job, less repeated work. That’s the core of making algorithms more efficient.

We’ve now seen two patterns:

Move unchanging work out of a loop (your sum(numbers) example).
Replace repeated scanning with a dict lookup (your players/scores example).

Both are about keeping the behavior the same, but cutting wasted steps.

Let’s level this up a bit with nested loops and Big O next (still in Python, small code).

Are you up for another challenge that focuses on reducing a nested loop?

Yes, give me the challenge

Alright, here’s boss battle #3 👾

You’re checking if any pair of numbers in a list adds up to 10:

Python

```
nums = [1, 4, 6, 3, 7, 9]

found = False
for i in range(len(nums)):
    for j in range(len(nums)):
        if i != j and nums[i] + nums[j] == 10:
            found = True

print("Any pair sums to 10?", found)
```

This works, but:

It checks both (i, j) and (j, i) (like 4+6 and 6+4)
It keeps looping even after found is already True

So there’s a lot of extra work.

First, just reasoning (no editing yet):

What’s one simple change you could make to reduce wasted work here:

either avoid checking both orders, or stop earlier once a pair is found?

Describe the idea in words, and we’ll turn it into code next.

stop when a pair is found

Yes, that’s a great efficiency move 💡

Step by step idea:

As soon as we find a pair that sums to 10, we don’t need to keep checking other pairs.
So once we set found = True, we should break out of the loops instead of continuing.

Visually, the pattern is:

Python

```
for ...:
    for ...:
        if condition:
            found = True
            break      # stop inner loop
    if found:
        break          # stop outer loop too
```

That improves the best case a lot (if a good pair appears early).

Want to try adding this “early exit” pattern in the editor yourself, or would you like me to sketch the full code first and then you mirror it?

Python

```
nums = [1, 4, 6, 3, 7, 9]
print("Before")
found = False
passes = 1
for i in range(len(nums)):
    for j in range(len(nums)):
        if i != j and nums[i] + nums[j] == 10:
            found = True
        print(passes)
```

```
        passes += 1
print("Any pair sums to 10?", found)

print("After")
found = False
passes = 1
for i in range(len(nums)):
    for j in range(len(nums)):
        if i != j and nums[i] + nums[j] == 10:
            found = True
        print(passes)
        passes += 1
    if found:
        break

print("Any pair sums to 10?", found)
```

Ron, this is excellent work 👏

You did something really smart here: you measured the work with passes before and after. That's exactly how to build intuition about efficiency.

Before: you got up to 36 passes (all i, j pairs)
After: you stopped at 6 passes, because you broke once found was True 🎯

So:

Same answer: Any pair sums to 10? True
Much less work in the best case (when a matching pair is found early)

One small improvement you could add (not required, just to think about):

Right now you break the outer loop when found is True, but the inner loop still runs to the end of that row before that. You could also break the inner loop immediately when you set found = True to cut even more work.

Want to tackle one more challenge where we refactor a nested loop into a single pass, or do you prefer to pause here and talk a bit about how these changes relate to Big O (O(n), O(n²))?

Try the single-pass challenge

Nice, let's do a clean single-pass one 🚀

Here's the situation:

You want to count how many numbers in a list are greater than 10.

Current (inefficient) code:

Python

```
nums = [3, 12, 5, 18, 7, 11]

count = 0
for i in range(len(nums)):
    for j in range(len(nums)):
        if nums[i] > 10:
            count += 1
            break

print("How many > 10?", count)
```

What it's doing:

For each i, it loops over all j, but it never even uses j in the if.
The inner loop is useless; we only care about nums[i].

Your goal:

Turn this into a single-pass solution: just one loop, no nested loop, same final count.

Do this in the editor:

Start from that code.
Remove the inner for j in range(len(nums)): loop.
Use a single for loop over the list to count numbers > 10.
Keep the same output format: How many > 10? X.

Run, then Submit code, and I'll check your single-pass version

Python

```
nums = [3, 12, 5, 18, 7, 11]

count = 0
for i in range(len(nums)):
    for j in range(len(nums)):
        if nums[i] > 10:
            count += 1
            break

print("How many > 10?", count)

count = 0
for i in range(len(nums)):
     if nums[i] > 10:
        count += 1

print("How many > 10?", count)
```

Beautifully done, Ron

You've got:

Original (nested loop, wasted work):
Python

```
count = 0
for i in range(len(nums)):
    for j in range(len(nums)):
        if nums[i] > 10:
            count += 1
            break
```

Single-pass, no useless inner loop:
Python

```
count = 0
for i in range(len(nums)):
    if nums[i] > 10:
        count += 1
```

Same result (3), but now:

Each element is checked once → work grows like O(n)
Before, there was a nested loop (even though you broke early), shaped like O(n^2) in the worst case

So this is a clean example of "refactor nested loop → single pass".

Quick check-in:

Do you feel clear on why the second version is O(n) and the first version is O(n²) in the worst case, or want me to walk that through with a tiny visual table?

I'm clear on it

Good, then we can push one level deeper 💪

So far you've:

Removed repeated work from loops
Swapped nested loops for cleaner single-pass code
Used a dict to avoid repeated scanning

You've basically beaten the main "efficiency bosses" for this lesson already 🎮

I've got one last mixed challenge that combines these ideas: you'll choose a better data structure and avoid a nested loop at the same time. Want to take that final boss, or would you rather wrap this lesson here and move on to the next one soon?

Let's try the final boss

Final boss coming up, Ron 😈🔥

You're managing which users are online in a chat app.

Current code:

Python

```
online_users = ["alice", "bob", "carol", "dave"]
events = [
    ("login", "erin"),
    ("logout", "bob"),
    ("login", "frank"),
    ("logout", "carol"),
]

# For each event, check if the user is currently online using a list
for action, user in events:
    is_online = False
    for name in online_users:
        if name == user:
            is_online = True
            break

    if action == "login" and not is_online:
        online_users.append(user)
    elif action == "logout" and is_online:
        online_users.remove(user)

print("Online now:", online_users)
```

Problems:

For every event, we scan the whole online_users list to see if user is online.
As online_users grows, this becomes a lot of repeated work.

Your mission in the editor ✏️:

Keep the same behavior and same final print (same users online at the end).
Replace the list-based "scan every time" approach with something faster in Python:
Use a dictionary that maps usernames → online status (True/False)

or a dictionary that maps usernames → exists (and you only care if the key is there).
For each event:
Use direct lookup (in or dict[...]) instead of scanning a list.
Update your structure accordingly.
You can still keep online_users as a list for the final print if you want, or derive it from your dict at the end.

Don’t worry about perfect style, just focus on:

no inner loop over online_users
using direct lookup instead

Run it, then Submit code, and we’ll talk through how much work you saved 🚀

Python

```
online_users = ["alice", "bob", "carol", "dave"]

event_dict = {
    "alice": "login",
    "bob": "logout",
    "carol": "login",
    "dave": "logout"
}
# For each event, check if the user is currently online using a list
online_list = []
for name in online_users:
    if event_dict[name] == "login":
        online_list.append(name)

print("Online now:", online_list)
```

Nice try, Ron — you’re thinking in the right direction (mapping names to statuses), but this version changed the meaning of the problem instead of processing the given events list step by step.

Original idea:

Start with online_users = ["alice", "bob", "carol", "dave"]
Then apply this sequence of events:
Python

```
events = [
    ("login", "erin"),
    ("logout", "bob"),
    ("login", "frank"),
    ("logout", "carol"),
]
```

Your code instead hard□coded a dict:

Python

```
event_dict = {
    "alice": "login",
    "bob": "logout",
    "carol": "login",
    "dave": "logout"
}
```

That’s not using the events at all — it’s just a different problem that happens to give ['alice', 'carol'].

Let’s simplify the “final boss” a bit and keep behavior very clear:

We’ll track who is online using a dict like:
online = {"alice": True, "bob": True, ...}

For each (action, user) in events, we'll:
check if the user is already in online with if user in online:
update online using direct lookups (no inner loop)

Here's a skeleton for you to fill in:

Python

```
online_users = ["alice", "bob", "carol", "dave"]
events = [
    ("login", "erin"),
    ("logout", "bob"),
    ("login", "frank"),
    ("logout", "carol"),
]

# 1) build dict from initial list
online = {}
for name in online_users:
    online[name] = True

# 2) process events with direct lookup (no loop over online_users)
for action, user in events:
    # TODO: update online based on action + current status
    # login: add if not already there
    # logout: remove if there

# 3) rebuild final list from dict keys
final_online = []
for name in online:
    final_online.append(name)

print("Online now:", final_online)
```

Your tasks in the editor:

Fill in the event-handling logic inside the for action, user in events: loop, using only dict operations (no scanning list).
Make sure the final behavior matches the original idea: start with 4 users, apply those 4 events in order.

Run it, then Submit code, and we'll look at the result together 🔍

Python

```
online_users = ["alice", "bob", "carol", "dave"]
events = [
    ("login", "erin"),
    ("logout", "bob"),
    ("login", "frank"),
    ("logout", "carol"),
    ("logout", "jack"),
]

#aded jack to prove it handles not removing non eistant users
# 1) build dict from initial list
online = {}
for name in online_users:
    online[name] = True

# 2) process events with direct lookup (no loop over online_users)
for action, user in events:
    if action == "logout" and user in online_users:
        online.pop(user)
    if action == "login":
        online[user] = action

    # TODO: update online based on action + current status
    # login: add if not already there
```

```
    # logout: remove if there

# 3) rebuild final list from dict keys
final_online = []
for name in online:
    final_online.append(name)

print("Online now:", final_online)
```

Ron, this is really good work — and I like that you added jack to test the edge case 🫶

You've basically done it; there are just two small fixes to make it cleaner and more correct:

Use the dict for membership, not the original list

Right now you have:

Python

```
if action == "logout" and user in online_users:
    online.pop(user)
```

That's checking user in online_users (the original list), but the true source of who's online is the dict online. We should check the dict:

Python

```
if action == "logout" and user in online:
    online.pop(user)
```

That way:

If the user isn't online, nothing happens (like your jack case).
We're not depending on the old list at all.

Store a simple boolean for logins

You did:

Python

```
if action == "login":
    online[user] = action   # stores "login" as the value
```

It works, but it's clearer and more consistent to store True for "online":

Python

```
if action == "login":
    online[user] = True
```

So the core event loop becomes:

Python

```
for action, user in events:
    if action == "logout" and user in online:
        online.pop(user)
    elif action == "login":
        online[user] = True
```

This still gives:

Python

```
Online now: ['alice', 'dave', 'erin', 'frank']
```

which is exactly right:

```
bob logged out
carol logged out
```

erin logged in
frank logged in
You've now:

Removed useless work in loops
Turned nested loops into single-pass code
Used dicts to replace repeated scanning
Added early exits to improve best-case time

That covers all the "make it more efficient" challenges for this lesson 🎯

If you'd like, I can:

quickly summarize how each change affected Big O (O(1), O(n), O(n²)), or
we can move on to the next lesson in your track.

What do you prefer?

Move to the next lesson

Done ☑ You've officially completed "Challenge: making an algorithm more efficient."

Quick recap of what you just mastered:

You spotted redundant work in loops and moved it out (like sum(numbers)).
You replaced nested scans with dict lookups, cutting a ton of wasted steps.
You turned nested loops into single-pass solutions when the extra loop wasn't needed.
You used early exit patterns (break) to improve best-case performance.
You reasoned about how changes affect Big O: when something is O(n²) vs O(n).
That's a big chunk of real-world algorithm thinking — this is how people actually speed up slow code at work 💻🔥

You're all set to continue to the next lesson from the Learn tab whenever you're ready.
If you hit anything confusing there, bring it back here and we'll unpack it step by step.

```
players = ["Ana", "Ben", "Cara", "Dan"]
scores = [10, 15, 8, 12]

requested = ["Cara", "Ana", "Dan"]

for name in requested:
    for i in range(len(players)):
        if players[i] == name:
            print(name, "has score", scores[i])
my_dict = {}
for i in range(len(players)):
    my_dict[players[i]] = scores[i]

# direct lookup, no nested loop
for name in requested:
    print(name, "has score", my_dict[name])


nums = [1, 4, 6, 3, 7, 9]
print("Before")
found = False
passes = 1
for i in range(len(nums)):
    for j in range(len(nums)):
        if i != j and nums[i] + nums[j] == 10:
            found = True
        print(passes)
        passes += 1
print("Any pair sums to 10?", found)

print("After")
```

```
found = False
passes = 1
for i in range(len(nums)):
    for j in range(len(nums)):
        if i != j and nums[i] + nums[j] == 10:
            found = True
            print(passes)
            passes += 1
            if found:
                break

print("Any pair sums to 10?", found)

nums = [3, 12, 5, 18, 7, 11]

count = 0
for i in range(len(nums)):
    for j in range(len(nums)):
        if nums[i] > 10:
            count += 1
            break

print("How many > 10?", count)

count = 0
for i in range(len(nums)):
     if nums[i] > 10:
        count += 1

print("How many > 10?", count)


online_users = ["alice", "bob", "carol", "dave"]
events = [
    ("login", "erin"),
    ("logout", "bob"),
    ("login", "frank"),
    ("logout", "carol"),
    ("logout", "jack"),
]

#aded jack to prove it handles not removing non eistant users
# 1) build dict from initial list
online = {}
for name in online_users:
    online[name] = True

# 2) process events with direct lookup (no loop over online_users)
for action, user in events:
    if action == "logout" and user in online:
        online.pop(user)
    if action == "login":
        online[user] = True


# 3) rebuild final list from dict keys
final_online = []
for name in online:
    final_online.append(name)
```